CS740: Blockchain Design Architecture

Task 3: Token Vesting with 6-Month Cliff

Submitted by

Samyak Gedam (252IS032)

Piyush K. Karn (252IS026)

Prithiraj Rabha (252IS028)

II Semester M-Tech CSIS

Submitted To: Dr. Sourav Kanti Addya

Department of Computer Science and Engineering

National Institute of Technology Karnataka

Surathkal, Mangalore-575025, India

2025–2026

Contents

1 Problem Statement **2**

2 Project Overview **2**

3 Tools and Technologies Used **2**

4 System Architecture Setup **2**

5 Technical Implementation **3**
5.1 Vesting Logic 3
5.2 Smart Contract Snippet 3

6 Execution Steps **3**

7 Execution and Verification Proofs **4**
7.1 Smart Contract Deployment 4
7.2 Cliff Verification 4
7.3 Initial Claimable Amount 5
7.4 Linear Vesting Progress 6
7.5 Further Time Progression 6
7.6 Final Withdrawal 7

8 Conclusion **7**

1 Problem Statement

The objective of this assignment is to implement a token vesting smart contract where funds are locked for an initial cliff period of 6 months. After the cliff, funds are released linearly over a period of 24 months. The beneficiary should only be able to withdraw the vested portion at any given time.

2 Project Overview

The project demonstrates a decentralized compensation mechanism using Ethereum smart contracts. The contract enforces a time-based restriction on fund withdrawal.

The cliff period ensures that no funds can be withdrawn before a fixed duration, preventing early exits. After the cliff, funds are released gradually using linear vesting, ensuring fairness and continuous incentive alignment.

For demonstration purposes, the cliff and vesting durations were reduced to seconds instead of months.

3 Tools and Technologies Used

- Ethereum Blockchain (Geth - Private Dev Mode)
- Remix IDE
- Solidity
- remixd (Local File Integration)

4 System Architecture Setup

The implementation was executed using:

- Local Ethereum blockchain using Geth developer mode
- Remix IDE connected via HTTP provider
- remixd to sync local files with Remix

5 Technical Implementation

5.1 Vesting Logic

The contract uses `block.timestamp` to track time. The release follows a linear progression after the cliff period.

$$\text{Vested Amount} = \frac{\text{Total Amount} \times (\text{Current Time} - \text{Start Time})}{\text{Vesting Duration}} \tag{1}$$

5.2 Smart Contract Snippet

```
// Key logic snippet
uint256 public constant CLIFF_DURATION = 0 seconds;
uint256 public constant VESTING_DURATION = 100 seconds;

function claimableAmount() public view returns (uint256) {
if (block.timestamp < startTime + CLIFF_DURATION) return 0;
}
```

6 Execution Steps

1. Start Geth in developer mode
2. Run remixd to connect local project folder
3. Open Remix IDE and connect to local blockchain
4. Compile and deploy the smart contract with ETH
5. Call `claimableAmount()`
6. Call `claim()` to withdraw funds

7 Execution and Verification Proofs

7.1 Smart Contract Deployment

The contract was successfully deployed with a 10 ETH initial balance. The deployment transaction was mined on the local Geth node, initializing the `startTime` state variable.

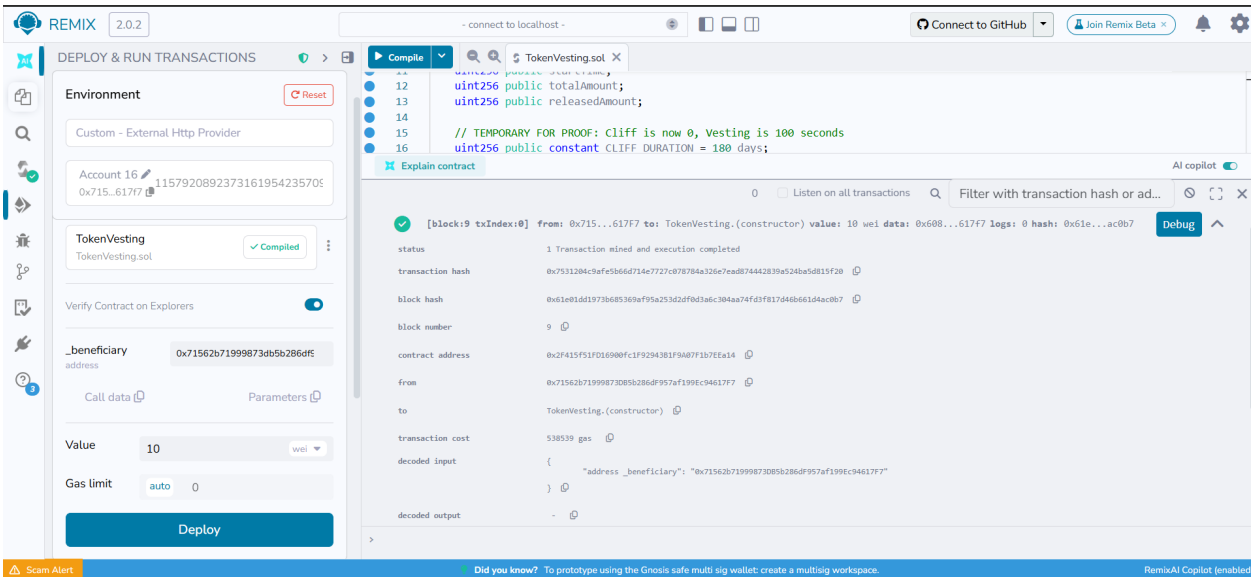

Figure 1: Smart contract deployed successfully

7.2 Cliff Verification

To verify the 6-month cliff constraint, a `claim()` transaction was attempted immediately following deployment. As shown below, the transaction was reverted by the EVM, proving that the cliff logic correctly blocks unauthorized early withdrawals.

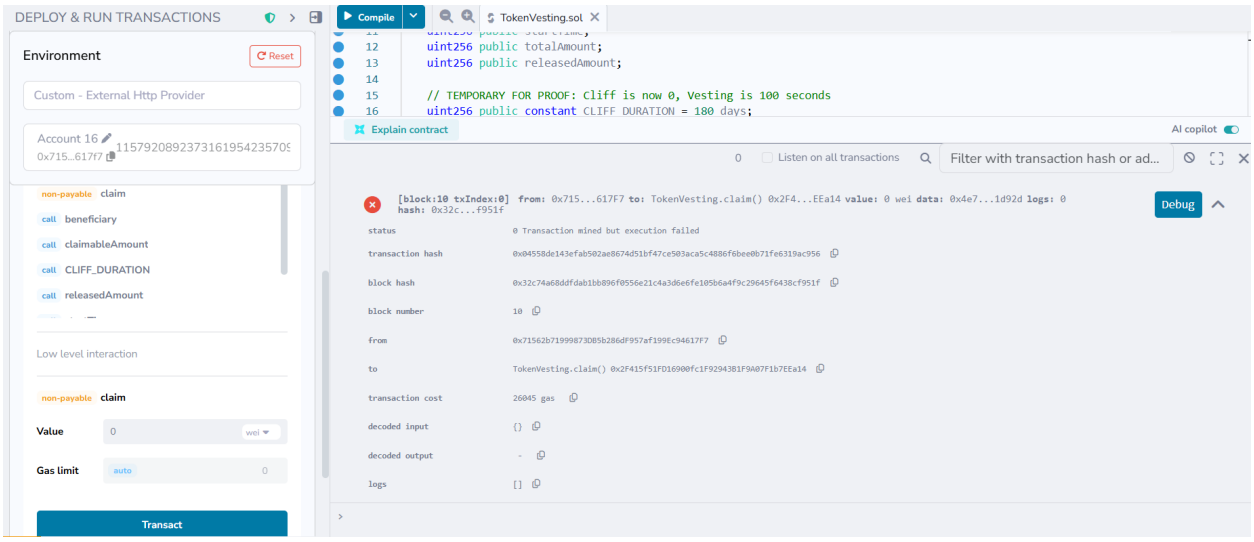

Figure 2: Transaction reverted before vesting period

7.3 Initial Claimable Amount

Immediately after deployment, the `claimableAmount()` function was called. As expected, the value returned was zero, confirming that no funds are available for withdrawal at the initial stage.

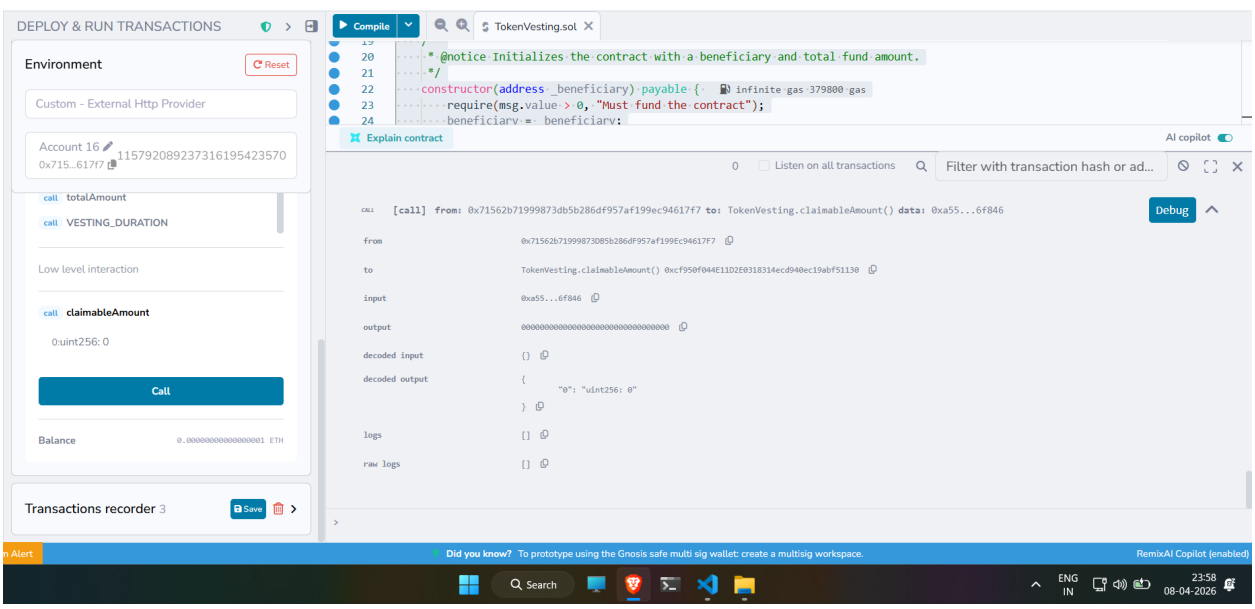

Figure 3: Initial claimable amount is zero

7.4 Linear Vesting Progress

After some time progression, the `claimableAmount()` function was called again. The returned value increased, demonstrating that funds are being released gradually according to the linear vesting logic.

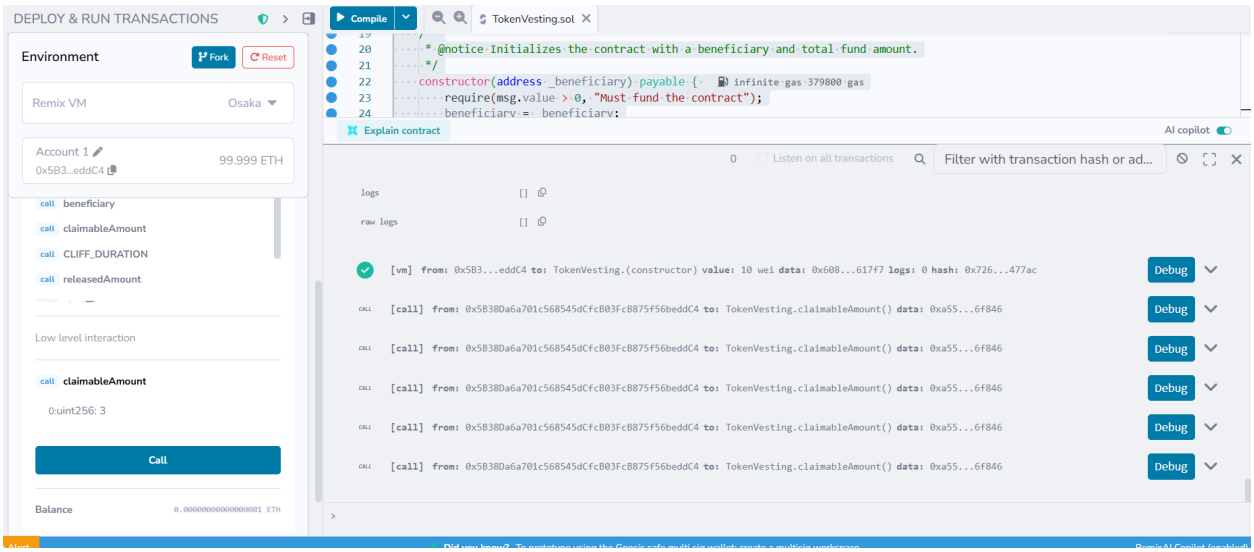

Figure 4: Claimable amount increasing due to linear vesting

7.5 Further Time Progression

With additional time elapsed, the claimable amount increased further, showing proportional release of funds over time.

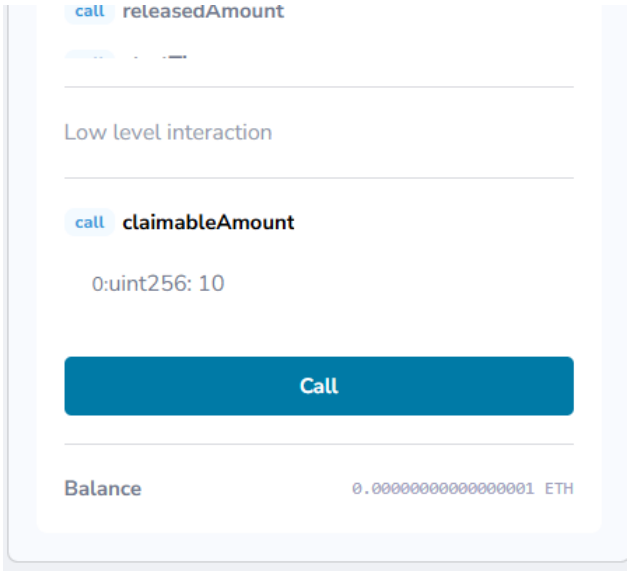

Figure 5: Claimable amount after further time progression

7.6 Final Withdrawal

Finally, the `claim()` function was executed successfully. The transaction was mined, gas was consumed, and the contract state was updated, transferring the vested amount to the beneficiary.

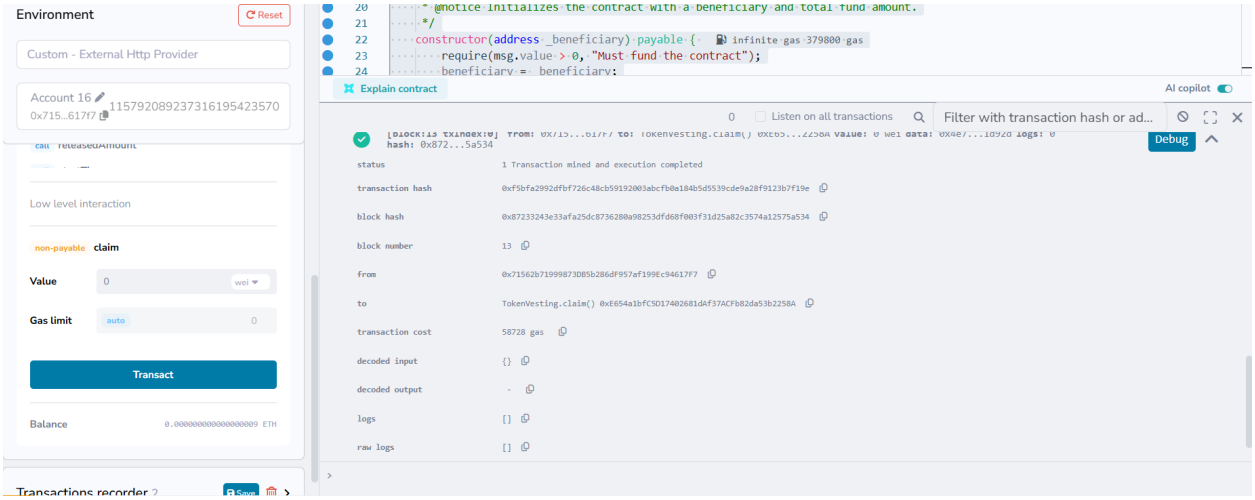

Figure 6: Successful execution of claim() function with transaction details

8 Conclusion

The smart contract successfully implements a token vesting mechanism with time-based restrictions. It ensures secure and gradual fund release without requiring a trusted intermediary. The implementation satisfies all assignment requirements and demonstrates practical blockchain-based compensation management.